```
            "test4", msg.record.key, msg.record.value),
          msg.committableOffset)                                    ❷
      }
      .toMat(Producer.committableSink(producerSettings, committerSettings))(
        Consumer.DrainingControl.apply)
      .run()
  ...
}
```

❶ ProducerMessage.Envelope

❷ PassThrough as an input for committableSink

`ProducerMessage.single` creates the `Envelope` and sets the `passThrough` with the `committableOffset` you get from `committableSource`. This way, you provide the necessary `Envelope[K,V,ConsumerMessage.Committable]` for the `committableSink`.

Other ProducerMessage methods

For completeness, it's useful to know that `ProducerMessage.single` is not your only option. You can use two other envelopes with a `committableSink`:

- `ProducerMessage.multi`—Produces a sequence of `ProducerRecords` instead of only one as `ProducerMessage.single`.
- `ProducerMessage.passThrough`—Only holds the `passThrough` and does not add any record to Kafka. Its only function is to commit the offset from the read records of a `committableSource`. This is like the `Committer.flow` you saw in the previous section or the `Committer.sink`.

Before we finish this section, let’s review by seeing a complete example that consumes and produces from and to Kafka using at-least-once semantics.